



PDF Download
3755881.3755926.pdf
04 March 2026
Total Citations: 0
Total Downloads: 996

Latest updates: <https://dl.acm.org/doi/10.1145/3755881.3755926>

RESEARCH-ARTICLE

Generating SysML Behavior Models via Large Language Models: an Empirical Study

YUAN WANG, Beihang University, Beijing, China

NING GE, Beihang University, Beijing, China

JIANGXI LIU, Beihang University, Beijing, China

ZHILONG CAO, Beihang University, Beijing, China

ZHEPING CHEN, Beihang University, Beijing, China

CHUNMING HU, Beihang University, Beijing, China

Open Access Support provided by:

Beihang University

Published: 27 October 2025

[Citation in BibTeX format](#)

Internetware 2025: the 16th International
Conference on Internetware
June 20 - 22, 2025
Trondheim, Norway

Conference Sponsors:
SIGSOFT

Generating SysML Behavior Models via Large Language Models: an Empirical Study

Yuan Wang
School of Software
Beihang University
Beijing, China
ayuan@buaa.edu.cn

Ning Ge*
School of Software
Beihang University
Beijing, China
gening@buaa.edu.cn

Jiangxi Liu
School of Software
Beihang University
Beijing, China
ZF2221306@buaa.edu.cn

Zhilong Cao
School of Software
Beihang University
Beijing, China
21371330@buaa.edu.cn

Zheping Chen
School of Software
Beihang University
Beijing, China
ZF2221113@buaa.edu.cn

Chunming Hu
School of Software
Beihang University
Beijing, China
hucm@buaa.edu.cn

Abstract

Model-driven development (MDD) is a mainstream approach in safety-critical domains, providing standardized modeling languages like SysML. SysML behavior models describe system dynamics and are widely used in aerospace, manufacturing, and IoT. However, manual modeling is inefficient and prone to quality issues, restricting MDD's practical adoption. The potential of LLMs in SysML behavior model generation and its challenges remain unclear, making it a key research topic. This empirical study evaluates LLMs in generating three types of SysML behavior models, focusing on performance and hallucinations. Our contributions are twofold: (1) constructing and publishing a dataset of 107 SysML behavior models spanning various domains; (2) analyzing hallucinations in LLM-assisted SysML behavior model generation from syntactic and semantic perspectives and proposing model-checking rules to mitigate them and enhance model quality. We analyze hallucinations in SysML behavior model generation, classifying them and exploring their possible causes. The evaluation results show that while the models generally meet syntactic requirements, they consistently lack semantic accuracy. Across both phases, LLMs achieve over 90% grammar accuracy. For semantic accuracy, the average F1-score for ACT reaches 95%, while SD drops to just 50%. These results demonstrate that while our model-checking rules effectively correct format and syntax, they are insufficient for addressing deeper semantic gaps. Overcoming these challenges requires advanced strategies, such as counterexamples and simulation traces, to provide optimal feedback. Additionally, model-checking in LLM-based generation is costly, and reducing this cost is another critical issue to address in the future.

*Corresponding author

Keywords

SysML behavior models, model generation, large language models, hallucination, model checking

ACM Reference Format:

Yuan Wang, Ning Ge, Jiangxi Liu, Zhilong Cao, Zheping Chen, and Chunming Hu. 2025. Generating SysML Behavior Models via Large Language Models: an Empirical Study. In *the 16th International Conference on Internetware (Internetware 2025)*, June 20–22, 2025, Trondheim, Norway. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3755881.3755926>

1 Introduction

Model-driven development (MDD) [63] is a key approach for designing safety-critical complex systems [21, 26, 34, 40]. SysML, a standardized systems modeling language, supports requirements, design, and verification and validation (V&V) [30, 59, 61, 76]. Its behavioral diagrams—use case (UCD), state machine (STM), activity (ACT), and sequence diagrams (SD)—capture system dynamics such as interactions, control flows, and state transitions [54]. UCDs specify "what" the system interacts with external actors rather than "how." STM, ACT, and SD capture internal behavior and control flow that underpin correctness and performance in safety-critical domains. This study focuses on these three diagram types. Despite SysML's strengths [68, 69], modelers need to build models by dragging and dropping graphical elements, a slow process even for simple cases, which raises the cost of MDD and discourages industrial adoption. Improving modeling efficiency while maintaining quality is crucial for broader adoption in safety-critical domains.

Recent advances in large language models (LLMs) are opening new avenues for automating software engineering [78]. LLMs perform strongly in natural-language understanding [20] and code generation [50], but their use for SysML model generation remains nascent yet promising [3]. Translating textual requirements into standardized behavioral models could substantially reduce manual efforts [23]. Apvrille and Sultan [3] conducted a preliminary evaluation in which ChatGPT-assisted Block Diagrams and STM were slightly better than student-built models. Tinnes et al. [70] explored the performance of LLMs in model generation and completion. Ferrari et al. [23] investigated LLM support for constructing SD. Across these studies, completeness and correctness with respect to the stated requirements remain recurring challenges. This



This work is licensed under a Creative Commons Attribution 4.0 International License. *Internetware 2025*, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1926-4/25/06
<https://doi.org/10.1145/3755881.3755926>

work investigates why LLMs succeed or fail at behavioral diagrams construction, catalogs the hallucination types they produce, and proposes integrating modeling-language knowledge and model checking to ensure high-quality automation.

We conducted an empirical study of LLMs performance and hallucinations when generating three SysML behavioral diagrams type across multiple domains. To enable this, we curated a dataset covering these diagrams. Because safety-critical sectors (e.g., aerospace) restrict data sharing [31, 48], we sourced models from open-source projects, SysML standards [53, 54], and authoritative books [28, 62]. The artifacts were heterogeneous and often lacked requirements, so we validated PlantUML compliance, SysML syntax and grammar, and requirement consistency, reverse-engineering requirements when absent. After 524 hours of collection and processing by modeling professionals, we compiled a dataset of 107 validated models.

We evaluate LLMs on generating SysML behavioral diagrams from natural-language requirements. **Phase I** designs a prompt template that enforces SysML grammar/semantics, PlantUML formatting, and requirements alignment, with supplied exemplar models. Over 70% of models were generated within 10 s, achieving >92% PlantUML format accuracy, >90% SysML grammar correctness, and >90% requirement consistency for activity diagrams. We analyzed hallucinations and devised model-checking rules. **Phase II** integrates these rules into a regeneration step, correcting most formatting (35/37) and grammar (22/25) issues but only modestly improved semantic hallucinations (31/72) and consistency (56/150). Finally, we examine causes of hallucination, propose richer feedback (e.g., counterexamples, simulation traces), and high cost of model checking as an ongoing challenge.

This study makes two key contributions to the research on LLMs and model-driven engineering. (1) **Publicly SysML behavior model dataset.** We release a curated dataset of 107 SysML behavior models (36 activity, 36 state machine, and 35 sequence diagrams) in PlantUML format. Drawn from peer-reviewed papers, textbooks, and open-source projects across multiple domains, each model is paired with its corresponding natural-language requirements. The dataset supports LLMs-enabled MDD research and is available here¹. (2) **Empirical evaluation of LLM performance and hallucinations in SysML behavior model generation.** Addressing the scarcity of systematic evaluations [23], we assess generation quality and categorize hallucination types. Model-checking rules effectively remove syntactic defects (grammar and format), but semantic issues persist, clarifying current limitations and pointing to future research directions.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 describes the construction of our model dataset. Section 4 outlines the research method. Section 5 and Section 6 present and analyze the empirical study results. Section 7 discusses threats to validity. Section 8 concludes the paper.

2 Related Work

SysML-based system modeling remains costly and time-consuming. Before LLMs, "intelligent" modeling largely relied on traditional

natural language processing and deep learning techniques. Gelhausen [27] and Bashir [9] proposed rule-based conversions from natural-language requirements to UML Class Diagrams. Ibrahim [36] and More [49] introduced domain ontologies to reduce ambiguity and improve concept recognition. Deva [19, 41] and Chmi [12] combined object-oriented analysis with syntactic parsing to generate UML class diagrams or SysML Block Definition Diagrams. These NLP-based methods depend on hand-crafted rules, input constraints, or bespoke architectures to cope with language informality, and thus suffer from low recognition accuracy, heavy input formalization, and limited domain knowledge - reducing generalizability across modeling scenarios.

Recent advances in LLMs have spurred interest in software modeling tasks. Table 1 surveys recent LLM-based modeling approaches. In requirements modeling, Jin et al. [39] systematically evaluated seven state-of-the-art LLMs on the CPSBENCH dataset to generate Cyber-Physical System (CPS) Problem Diagrams from natural language requirements. Results show promise, but limited domain-knowledge integration and frequent hallucinations hinder practical adoption, constraining applicability in industrial settings that demand high-precision specifications.

In design model generation, Conrardy et al. [15] showed GPT reliably converts hand-drawn UML Class Diagrams to PlantUML format, but syntactic and semantic errors grow with diagram complexity. Cámara et al. [16] reported persistent syntactic defects and semantic ambiguities in GPT-generated textual UML models, especially for large-scale diagrams (>12 classes) or complex concepts. Ahmad et al. [1] evaluated GPT as an architecture assistant (DevBot) in human-AI co-design and found strategy biases that conflicted with specific requirements. Wang et al. [72] found LLMs help UML novices identify basic elements but struggle with inter-class relationships and complex semantics, necessitating human supervision. Bader et al. [6] automated XMI code generation for UML Component Diagrams but noted reliability issues in medium-to-high complexity tasks due to XMI verbosity and context window limitations. Ferrari et al. [23] showed GPT can produce standard-compliant UML Sequence Diagrams from natural language, improving consistency and clarity yet falling short on behavioral completeness and scenario coverage. Apvrille et al. [4] integrated iterative model-checking feedback into the TTool-AI framework to enhance SysML Block and Internal Block Diagram generation, though effectiveness was constrained by LLM context processing limits. Tinnes et al. [70] formalized software model evolution using change graphs, automating UML/SysML/Ecore model completion via GPT, historical model data, and Retrieval-Augmented Generation (RAG). Experimental results show that LLMs perform well in model evolution tasks, though noise and lack of context often cause deviations from expected results.

In conclusion, while existing research focuses on model generation and completion, no prior work has systematically investigated LLMs' effectiveness and hallucinations in SysML behavior model generation. This study fills that gap with rigorous empirical methodologies.

¹https://drive.google.com/drive/folders/10eo8KDqIBlkQZxPpPCB7R3-aBQZ7Rsm6?usp=drive_link

Table 1: Related Work on LLMs-Assist Models Generation

Task	Model Type	Ref	Advantages	Limitations	Year
Requirements model generation	Problem Diagram	[39]	Leverages LLMs to extract entities and interaction semantics from CPS requirements descriptions, enabling automated Problem Diagram construction to enhance modeling efficiency	LLMs demonstrate limited accuracy in entity recognition and diagram modeling, with an average recall rate of approximately 60%.	2024
Design model generation	UML Class Diagram	[15]	Utilizes GPT to convert hand-drawn UML Class Diagram images into PlantUML models, significantly accelerating the conversion process	Struggles with complex diagrams; generates syntax errors in class relationships; produces unexpected outputs for semantically ambiguous inputs	2024
	UML Class Diagram	[18]	Employs single-prompt interactions to generate UML Class Diagrams from natural language requirements, achieving comparable grammar quality to human counterparts	LLMs exhibit inferior semantic quality and solution correctness compared to human-generated models	2024
	UML Class Diagram	[16]	Generates textual UML models for small-scale systems (10-12 classes), demonstrating high accuracy in OCL constraint handling	Frequent semantic errors in large models (>12 classes), including misrepresented aggregation & association relationships; requires iterative refinements introducing new errors	2023
	UML Class Diagram & Component Diagrams	[1]	Implements GPT as DevBot for collaborative architecture design, translating functional requirements into architectural specifications through dialog	Generates inconsistent outputs for identical queries; exhibits architectural bias from training data in large-scale systems	2023
	UML Class & Use Case & Sequence Diagram	[72]	Provides UML novices with modeling guidance using quantifiable quality metrics, showing potential in optimization	Low accuracy in relationship recognition necessitates manual revisions by modelers	2024
	UML Component Diagram	[6]	Generates XMI-formatted UML Component Diagrams, reducing technical barriers for non-experts	Context window limitations affect correctness; struggles with XMI element ID uniqueness	2024
	UML Sequence Diagram	[23]	Automates Sequence Diagram generation from natural language requirements with strong standards compliance	Fails to resolve ambiguous requirements; performs poorly in domain-specific behavioral representations	2024
	SysML Use Case & Block Definition Diagram	[67]	Integrates TTool framework with GPT for automated inconsistency detection/repair across diagrams	Requires manual correction for naming conflicts and requirement mismatches	2024
	SysML State Machine Diagram & Block Definition Diagram	[4]	Implements TTool-AI framework with error-detection feedback loops for model optimization	Context window limitations necessitate multiple iterations for complex systems	2024
Model completion	SysML/BPMN/Ecore	[70]	Combines retrieval-augmented generation (RMAC) with model history to achieve 62.30% semantic accuracy in industrial datasets	Generates structurally/ semantically invalid completions due to context/domain knowledge gaps	2024

3 Dataset Construction

3.1 Status of Missing SysML dataset

To identify relevant datasets, we searched English and Chinese literature via Google Scholar [29] and CNKI [52], and explored open-source projects on GitHub [46]. Only three studies mention SysML datasets [11, 66, 81]. Spangelo et al. [66] used a SysML model repository from a train product line in Magic Draw, containing 100 annotated models for training named entity recognition [11]. Unfortunately, the datasets from this work, as well as those from the other two studies, have not been made publicly available.

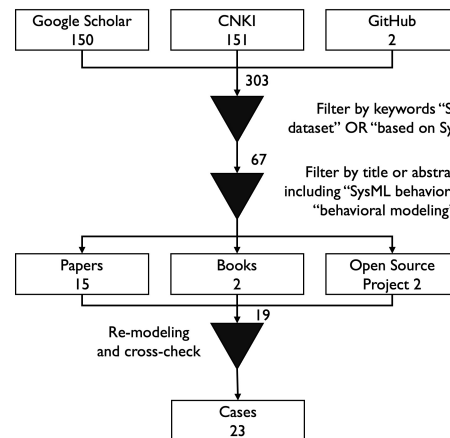
3.2 Dataset Construction Group

We have two groups: G_Search for data collection (G_Search) and G_Model for data processing & modeling. G_Search, with four senior undergraduates and one master's student, handled data acquisition. G_Model, comprising one senior undergraduate, two master's students, and two doctoral candidates, focused on data refinement and model construction. All members had software engineering backgrounds and MDD expertise, each with over 100 hours of modeling experience in system/software projects, ensuring high-quality data collection, processing, and model development.

3.3 Data Collection and Processing

The data collection and processing follow the framework outlined in Figure 1.

Data Collection Criteria. We built SysML behavior models across various domains, including aerospace, rail transportation,

**Figure 1: SysML behavior model dataset construction**

and IoT, each with corresponding requirement descriptions or models. To ensure data quality, we established the following inclusion (IC) and exclusion criteria (EC): *IC1*: The source (article, book, or open-source project) uses SysML behavior models as study cases. *IC2*: The source contains clear graphical models. *IC3*: The models conform to the SysML v1.6 specification. *EC1*: Non-peer-reviewed literature. *EC2*: Sources without full-text access.

Data Collection. G_Search searched Google Scholar and CNKI using the keywords "SysML Dataset" OR "based on SysML" and also explored open-source projects on GitHub. This process yielded 303 results: 148 English papers, 2 English books, 151 Chinese papers,

and 2 open-source projects. An overview of these cases is provided in Table 2.

Data Processing. The collected models were primarily pre-built SysML diagrams, mostly available only as images. To ensure format consistency, G_Model reconstructed all cases. Even for models with source files, refinement was needed to meet dataset standards. We used PlantUML [56] with SysML v1.6 as the modeling language. Each G_Model member focused on a specific behavior diagram type, using natural language requirements when available. For undocumented cases, we analyzed model structures and behaviors to infer implicit requirements and created corresponding descriptions. All reconstructed models were cross-validated against the original sources for consistency.

Requirements Specification. To ensure high-quality requirement descriptions, we developed a requirements specification template. Figure 2 presents a state machine diagram and its requirement description for a Hybrid Sport Utility Vehicle case, along with the specification template.

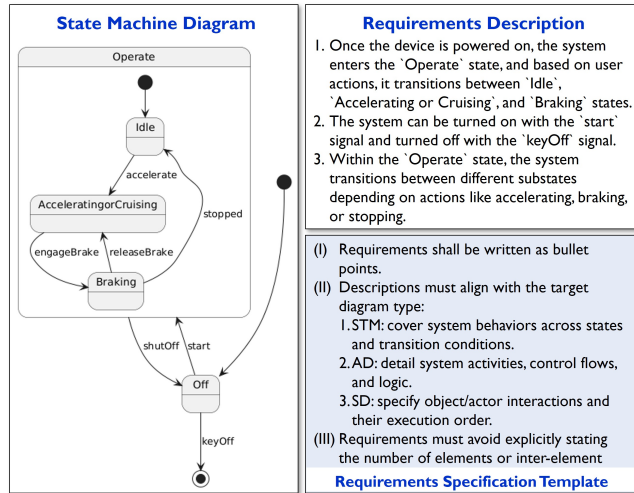


Figure 2: State machine example and its requirements

3.4 SysML Behavior Model Dataset

The dataset comprises 107 SysML behavior models: 36 activity diagrams, 34 state machine diagrams, and 35 sequence diagrams. These models span five domains: equipment manufacturing (29%), rail transportation (24%), IoT (19%), aerospace (19%), and military (9%). An overview of the dataset is presented in Table 2.

4 Design of Empirical Study

4.1 Research Questions

RQ1: How effective are LLMs in generating SysML behavior models, and what common hallucinations occur in their outputs? Our goal is to assess the effectiveness and hallucinations of LLMs in generating SysML behavior models. We will evaluate the quality of generated models, generation time, and overall performance using different LLMs, while also analyzing the types of hallucinations observed in the results.

RQ2: How does model checking feedback help reduce hallucinations in SysML behavior models generated by LLMs?

We aim to address hallucinations in behavior models generated by LLMs. Based on the results from RQ1, we propose syntax and semantic checking rules and integrate the feedback into the prompt to guide model regeneration. Additionally, we will analyze how this feedback impacts the quality of models generated by different LLMs.

4.2 Evaluation Process

Figure 3 summarizes two-phase experimental design. **Phase-I** evaluates LLMs on SysML behavioral diagrams generation and identifies potential hallucinations. We employ a RAG component to retrieve relevant PlantUML format and SysML v1.6 specification text, and we structure requirements plus auxiliary context into a prompt template. Outputs are assessed by time-to-generate, PlantUML formatting accuracy, SysML grammar conformance, semantic consistency (F1-score), and hallucination taxonomy. Hallucinations are grouped into four types: PlantUML formatting errors, SysML grammar errors, SysML semantic inconsistencies, and requirements semantic inconsistencies.

Phase-II corrects hallucinations via regeneration guided by model-checking feedback. Detected issues and their associated rules are processed by the same validator as in Phase 1. The resulting feedback is injected into a revised prompt and sent to the LLM for regeneration. Regenerated models are evaluated on the same four dimensions. To avoid contamination, each regeneration runs in a fresh LLM session. Phase-II implements four classes of model-checking rules (detail below).

(1) **PlantUML format checking.** Validates conformance to PlantUML syntax/format rules. Results are reported automatically by the PlantUML model checker.

(2) **SysML grammar checking.** Assesses adherence to SysML grammar. Because PlantUML lacks a SysML grammar checker, we manually compare each item against the standard and record the errors.

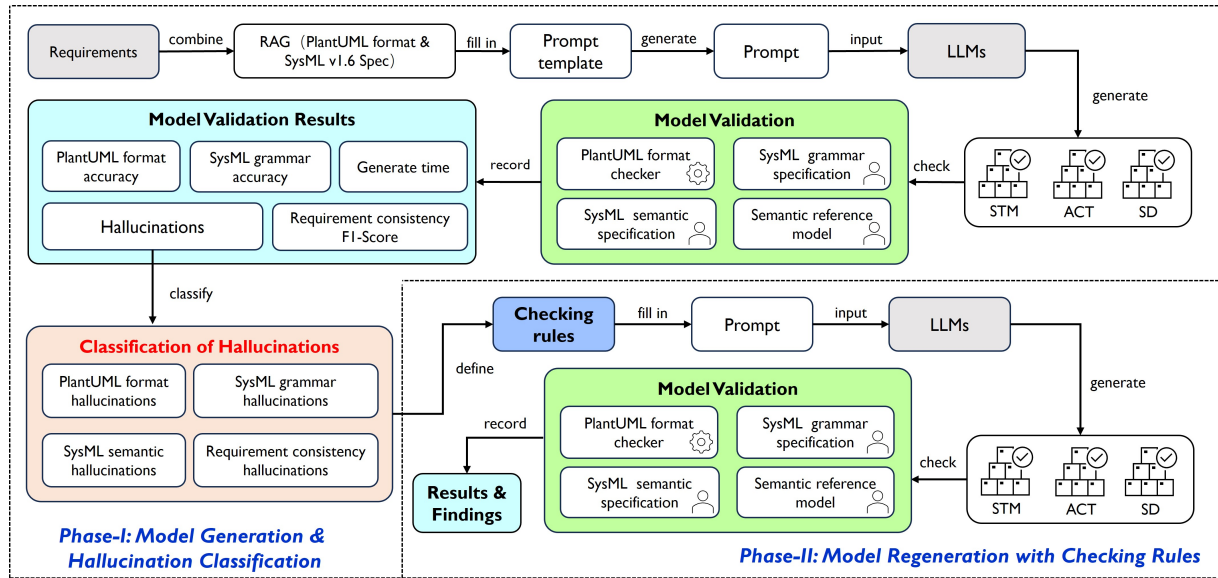
(3) **SysML semantic checking.** Assessed compliance with semantics defined in the SysML standard. As PlantUML does not support semantic validation, we manually check the model against 55 semantics and log violations.²

(4) **Requirements semantic checking.** We define this as evaluating semantic consistency between the generated model and a reference model. In code generation, syntactic/structural accuracy is usually measured via static analysis (e.g., AST matching [2], CodeBLEU [60], ROUGE-L [42], METEOR [7], ChrF [57]), while functional correctness uses metrics such as Pass@k [13]. For model generation, grammar accuracy is split into PlantUML format accuracy (Acc_p) and SysML grammar accuracy (Acc_g), while functional correctness typically requires simulation-based validation [25, 45]. In this study, we assume the reference model is semantically correct and compute the F1-score between the generated and reference models to quantify requirement semantic alignment.

²Detailed grammar and semantics are available in the open-data resource.

Table 2: Model Dataset Overview

Domain	Case Name	Abbreviation	Source	N#ACT	N#STM	N#SD
Manufacturing	Aluminum Extrusion Press System	AEPS	Paper[75]	2	0	0
	Diesel Engine System	DES	Paper[43]	3	0	0
	High-Level Driving Control System	HLDCS	Paper[44]	0	3	0
	Flexible Manufacturing System	FMS	Paper[58]	3	0	2
	Distilled Water Production System	DWPS	Practical Guide to SysML [62]	6	2	1
	Hybrid Sport Utility Vehicle	OS-HSUV	SysML Official[53]	2	1	2
Rail Transit	EMU Traction Converter	EMUTC	Paper[73]	1	0	1
	High-Speed Train Bogie System	HSTBS	Paper[32]	1	1	1
	Train Control Interlocking Integrated System	TCIIS	Paper[82]	7	0	0
	Railway Crossing Control System	Book-RCCS	Real-Time Software Design for Embedded Systems [28]	0	3	4
	Light Rail Control System	Book-LRCS	Real-Time Software Design for Embedded Systems [28]	0	9	8
Internet of Things	Drone Swarm Combat System	DSCS	Paper[80]	2	2	0
	Botanical Garden Intelligent Guide System	BGIGS	Paper[74]	0	0	3
	Microwave Oven Control System	Book-MOCV	Real-Time Software Design for Embedded Systems [28]	0	6	5
	Pump Control System	Book-PCS	Real-Time Software Design for Embedded Systems [28]	0	4	0
	Digital Camera System	DCS	Paper[8]	0	1	0
	Automated Teller Machine	ATM	Paper[8]	0	1	0
Aerospace	Airborne Communication System	ACS	Paper[38]	0	0	3
	Airborne Microstrip Antenna System	AMAAS	Paper[10]	4	0	0
	Pilot-in-the-loop Civil Aircraft Simulation System	PCASS	Paper[14]	1	1	1
	Spacecraft Launch Control System	Book-SLCS	Practical Guide to SysML [62]	1	1	1
Military	Airborne Antenna Array Simulation System	AAASS	Paper[79]	1	0	3
	Combat Scenario Monitoring Simulation System	CSMSS	Paper[33]	2	1	0
Total				36	36	35

**Figure 3: Study pipeline**

4.3 Dataset Selection

We divided the dataset into three categories based on behavior model types and randomly selected ten models from each. To represent model scale intuitively, we introduced the concept of a grammar point (GP), denoting the total number of SysML syntax elements in a model.

DEFINITION 1. *Nodes of Behavior Model*

- $N_{STM} = \{State, Vertex, Pseudostate, Region\}$
- $N_{ACT} = \{ActivityNode, Variable\}$
- $N_{SD} = \{Lifeline, CombinedFragment\}$

DEFINITION 2. *Edges of Behavior Model*

- $E_{STM} = \{Transition\}$
- $E_{ACT} = \{ActivityEdge\}$
- $E_{SD} = \{Message\}$

DEFINITION 3. *Behavior Model*

A behavior model M is a 2-tuple $\langle n_t, e_t \rangle$ where n_t is N_{STM} or N_{ACT} or N_{SD} ; e_t is E_{STM} or E_{ACT} or E_{SD} .

The grammar point for the model of type t is computed as

$$GP_t = |n_t| + |e_t| \quad (1)$$

For example, the GP value for the state machine diagram in Figure 2 is 17, consisting of 2 *InitialState*, 1 *FinalState*, 4 *SimpleState*,

1 *CompositeState*, and 9 *Transition*. The complexity of the experiment dataset is presented in Table 3.

Table 3: Scale of SysML Models

Type	Name	GP	Type	Name	GP	Type	Name	GP
STM	HLDCS	11	ACT	Book SLCS	15	SD	Book PCS	10
	HSTBS	13		CSMSS	16		Book PCS	12
	Book PCS	15		DWPS	21		Book PCS	13
	OS HSUV	21		DSCS	22		BGIGS	15
	TCIIS	21		HSTBS	24		Book SPCS	18
	MOCV	25		TCIIS	25		Book PCS	18
	DSCS	26		DSCS	26		EMUTC	22
	HLDCS	30		FMS	37		ACS	25
	DCS	34		AEPS	40		FMS	29
	HLDCS	90		ACS	76		FMS	46

4.4 Modeling Tool Selection

Table 4 summarizes our survey of 10 SysML modeling tools, evaluating their support for SysML v1.6, data format openness, and accessibility. Only Papyrus and PlantUML satisfied all criteria. We selected PlantUML for our experiments due to its simpler and more intuitive format compared to Papyrus’s XMI representation. Although the selected PlantUML tool is user-friendly and complies with the SysML v1.6 specification, it lacks built-in capabilities for SysML syntax and semantic checking. Consequently, all validations must be performed manually, potentially limiting the comprehensiveness and automation of the research. Future studies are recommended to utilize more powerful SysML modeling tools or develop specialized automated model-checking tools to enhance the comprehensiveness and accuracy of evaluations.

Table 4: SysML Modeling Tools

Name	Version	SysML Spec v1.6	Data Format	Availability
SinlaboreRT [64]	6.5	No for SD	Public	Business
Trufun [71]	v2024	No for STM	Unknown	Business
Software Ideas Modeler [47]	14.85	Only for v1.5	Partially public	Business
Enterprise Architect [5]	17.0	Only for v1.5	Partially public	Business
Gaphor [24]	2.27	Partially supported	Public	Free
Visual Paradigm [55]	17.2	Partially supported	Partially public	Business
Rhapsody [35]	10.0.1	Partially supported	Partially public	Business
MagicDraw [17]	v2022	Fully supported	Partially public	Business
Papyrus [22]	6.7.0	Fully supported	Public	Free
PlantUML [56]	v2025	Fully supported	Public	Free

4.5 LLMs Selection

Table 5 lists the six LLMs used in our experiments, including closed-source models (e.g., GPT-4o, GPT-4) and open-source models (e.g., Llama3.1, DeepSeek-v3), all representing the latest versions from major providers. Closed-source models were accessed via official APIs, and open-source models via Nvidia’s API [51]. To ensure a fair comparison, all models were configured with identical parameters (e.g., temperature = 0). Although Claude 3 Haiku supports a 200,000-token context window, our experiments remained within this limit to eliminate context length as a confounding factor.

4.6 Prompt Template Design

Figure 4 illustrates the prompt for generating an activity diagram. The prompt P consists of five components: role R , instruction I , requirements Req , sample prompt S , and error information E .

Table 5: Evaluated LLMs in Study

Type	Name	Version	Context	Publisher
Close-source	GPT-4	GPT-4-Turbo	128,000 tokens	OpenAI
	GPT-4o	GPT-4o-2024-11-20	128,000 tokens	OpenAI
	Kimi	Moonshot-v1	128,000 tokens	Moonshot AI
	Claude	Claude 3 Haiku	200,000 tokens	Anthropic
Open-source	Llama	Llama3.1	128,000 tokens	Meta AI
	DeepSeek	DeepSeek-v3	128,000 tokens	deepseek

The notation $\$$ represents variable configuration items, allowing flexibility for different modeling tasks.

Model Generation Prompt
Role: You are an expert in SysML modeling ## Instruction: Please help me to build an $\{activity\}$ model as per the requirement ## Requirements: 1 - The output data format must comply with $\{PlantUML\}$ 2 - Model code needs to be output, no other content needs to be output 3 - $\{I\}$. This process is mainly implemented by the TIS of the local station, the TIS of the neighboring station (TCC) and the ZPW-2000 track circuit. 2. Three partitions are set in the activity diagram model of the running direction initialization process. ## Sample: (PlantUML Language Reference Guide (1.2025.0) 33 / 6061.37 Skinparam 1 Activity Diagram <pre>@startuml start if (multiprocess?) then (yes) fork :process 1; fork again :process 2; end fork else (no) :logic 1; :logic 2; endif @enduml</pre> SysML Specification v1.6 (OMG) (1.2025.0) 13 / 15231.31 Skinparam 1 Activity Diagram An Activity is a Behavior specified as sequencing of subordinate units, using a control and data flow model. The execution of one ActivityNode within an Activity may affect, and may be affected by, the execution of other ActivityNodes in the Activity. ## Error: $\\$ <pre>(end fork: Cannot find fork Fork nodes cannot contain only one sub activity }</pre>

Figure 4: Prompt example of activity model generation

Role(Role): The role is “You are an expert in SysML behavioral modeling,” which helps reinforce the professionalism of the LLM’s responses within the modeling context.

Instruction(I): The instruction defines the type of behavioral model to generate and the rules to follow. This task is configurable; for instance, in Figure 4, the LLM is instructed to create an activity diagram based on the given requirements.

Requirements(Req): Requirements play a key role in guiding the LLM to complete various modeling tasks. They are divided into three main parts: (1) **Output Format:** We built our dataset based on the PlantUML’s data format, providing corresponding syntax instructions in the sample. (2) **Output Content:** We direct the LLM to output only the model content, ensuring focus on high-quality generation. (3) **Requirements Descriptions:** We provide requirement descriptions for the model, guiding the LLM to generate the corresponding behavioral model.

Sample(S): The sample provides the PlantUML format with RAG support, including SysML syntax and semantics, ensuring the LLM has complete information for model generation.

Error(E): We feed model checking results as error messages to the LLM, highlighting issues to improve model accuracy.

We used greedy sampling [65] with a temperature setting of 0. Greedy sampling ensures that the LLM selects the most likely result at each step, thus avoiding the randomness associated with other sampling methods and guaranteeing the consistency and reliability of generated responses.

4.7 Evaluation Metrics

We designed four evaluation metrics in Table 6: Model Generation Time (T_G), PlantUML Format Accuracy (Acc_P), SysML Grammar Accuracy (Acc_S), and Semantic Consistency (F1-score).

Table 6: Metrics of LLM Modeling Capability

Metrics	Explanation	Unit
T_G	$T_g - T_p$, where T_p is the time when the prompt is fed to the LLM and T_g is the time when the model is generated.	second
Acc_P	N_c^S / N_a^S , where N_c^S is the number of correct PlantUML sentences and N_a^S is the total number of sentences in the model.	percentage
Acc_S	N_c^{GP} / N_a^{GP} , where N_c^{GP} is the number of correct SysML grammar point and N_a^{GP} is the total number of grammar point in the model.	percentage
F1-score	$2 \times TP / (2 \times TP + FN + FP)$, where TP is the number of GPs correctly identified, FP is the number of GPs that do not exist in the reference model, and FN is the number of GPs incorrectly identified.	percentage

5 Results and Analysis of RQ1

5.1 Model Generation Time in Phase-I

G_T in Phase-I is shown in Figure 5. Timing was recorded from the moment the prompt was sent to the LLM until the full output was received. Based on our observations, behavioral model generation typically takes 6–15 seconds in real-world scenarios. Over 70% of models were generated within 10 seconds.

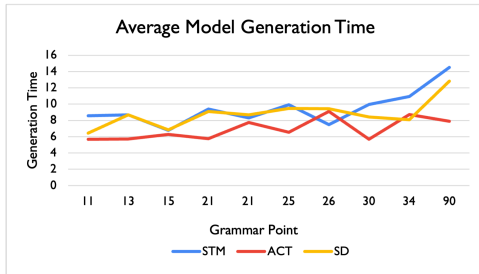


Figure 5: Model Generation Time (T_G) in Phase-I

5.2 Syntactic Accuracy in Phase-I

Figure 6 and Figure 7 show the results for Acc_P and Acc_S . Both metrics exceed 90% across all behavioral models. Notably, Acc_P for ACT (>97%) and SD (>98%) approaches 100%, as do Acc_S for STM (>98%) and ACT (>99%).

Prior research [77] has shown that LLMs often struggle with nested structures (NS) and field constraints (FC) to follow complex, domain-specific formats. In our research context, nested structure refers to the hierarchy or containment relationship in the SysML behavior model, while field constraints are the rules that define the data types that fields should contain, such as requiring elements to be SysML keywords. The results in Table 7 show that providing additional context via RAG significantly improves syntactic accuracy. These findings suggest that detailed PlantUML formatting instructions and SysML grammar descriptions can effectively mitigate LLMs' inherent syntactic reasoning limitations in behavior model generation.

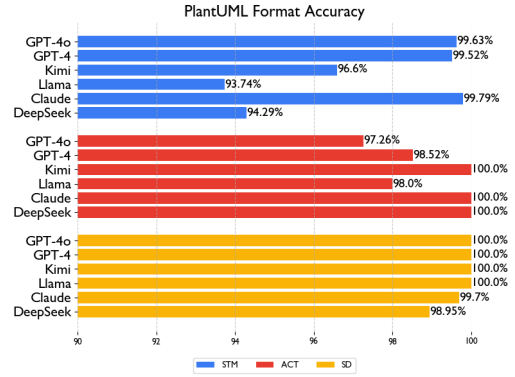


Figure 6: PlantUML Format Accuracy (Acc_P) in Phase-I

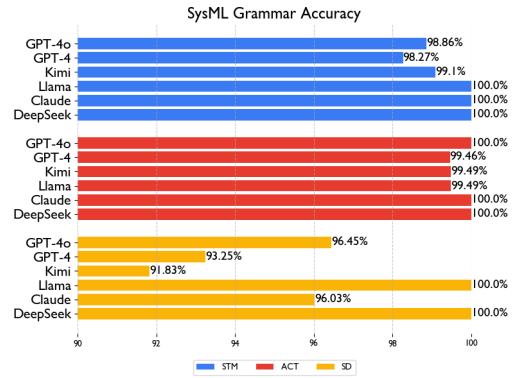


Figure 7: SysML Grammar Accuracy (Acc_S) in Phase-I

Table 7: Statistics on Acc_S

Model Type	N#NS	N#FC	$Acc_S@NS$	$Acc_S@FC$
STM	70%	20%	97.26%	69.29%
ACT	40%	40%	99.31%	97.49%
SD	60%	100%	99.77%	50.02%

5.3 Semantic Consistency in Phase-I

Figure 8 reports semantic consistency F1-score across the three behavioral diagram types. ACT attains the highest mean (97.49%), indicating strong alignment with reference models. STM averages 69.29% (65.17% - 80.27%), suggesting LLMs capture key states and transitions but with incomplete coverage. SD is lowest at 50.02%, recovering only half of interaction structures and thus proving most challenging for LLM-based generation.

We attribute these gaps primarily to structural complexity. ACT typically follows linear or lightly branched flows suited to sequential reasoning, yielding the top score. STM requires extracting salient states and transitions from requirements - moderate difficulty, with models showing stronger inductive ability (e.g., Claude) performing better. SD coordinates multiple objects exchanging strictly ordered messages and demands deeper context; without explicit cues, LLMs struggle to infer accurate interactions and message names, driving the sharp drop. In general, the greater structural complexity and requirement dependence correlate with lower semantic alignment.

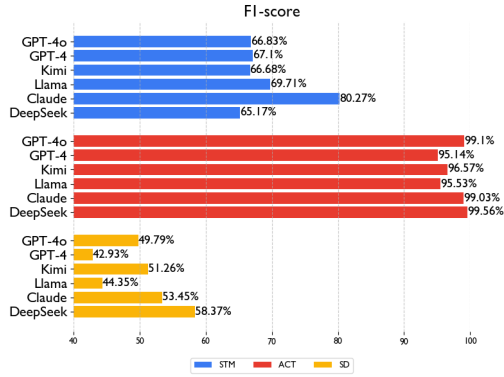


Figure 8: F1-score results in Phase-I

5.4 Summary of Hallucinations

Hallucination refers to LLM-generated content that deviates from the source or lacks logical coherence [37]. We define hallucinations as outputs that do not conform to the input requirements, and categorize them into three types: PlantUML format errors, SysML grammar & semantic errors and requirements inconsistencies.

Table 8 summarizes the PlantUML format hallucinations. Among the 37 recorded cases, Non-Existent Elements account for the majority (81.08%), such as using undefined elements like “stm” in STM. Format Errors make up 10.81%, including incorrect combinations like “opt” and “else” in SD. The remaining cases involve less frequent issues such as Element Not Closed and Object Naming Errors.

Table 8: PlantUML Format Hallucination

Type	Number	Percentage
Non-Existent Element	30	81.08%
Format Error	4	10.81%
Element is not Closed	3	8.11%
Total	37	100.00%

Table 9 lists the SysML grammar and semantic hallucinations. For grammar hallucinations, common issues include: Blocks incorrectly generated as Actors in SD, Transitions not connecting two states in STM, Missing internal States in Composite States, Fork Nodes with only a single outgoing branch in ACT. For semantic hallucinations, observed problems include: Missing Regions in Composite States and omitted PseudoStates (e.g., Join or Fork) in STM, Missing ActivityFinalNodes, ReduceActions, and meaningless ActivityPartitions in ACT, frequent omission of DestructionOccurrenceSpecifications for Message terminations in SD.

Table 10 summarizes the requirements inconsistency hallucinations. For STM, a total of 23 hallucinations were classified as Model Inaccuracy, including missing States, Transitions, FinalStates, Composite States, and incorrect Transitions. For ACT, 27 hallucinations were identified. About two-thirds were caused by model inaccuracies, like missing ActivityNodes, parallel structure errors, and missing InitialNodes. The remaining third was due to model over-specification, mainly extra ActivityNodes not specified in the requirements. For SD, 100 hallucinations were detected, all under Model Inaccuracy. Common issues included incorrect Message

Table 9: SysML Grammar & Semantic Hallucination

Classification	Type	Number	Percentage
Grammar	STM: Transition syntax error	4	16.00%
	STM: Composite State syntax error	2	8.00%
	ACT: Fork syntax error	1	4.00%
	SD: Block syntax error	18	72.00%
	Total	25	100.00%
Semantic	STM: missing Regions	20	27.78%
	STM: missing PseudoState	5	6.94%
	ACT: normal nodes as ActivityFinalNode	1	1.39%
	ACT: missing ReduceActions	1	1.39%
	ACT: meaningless ActivityPartitions	1	1.39%
	SD: missing DestructionOccurrenceSpecification	44	61.11%
	Total	72	100.00%

types (e.g., wrong synchronous/asynchronous calls), misuse of CombinedFragments (e.g., conditional segments), missing Messages (required interactions omitted), and Lifeline problems (either missing participants or incorrect identification).

Table 10: Requirement Inconsistency Hallucination

Model	Classification	Type	Number	Percentage
STM	Model Inaccuracy	Missing State or Transition	7	30.43%
		Incorrect Transition	6	26.09%
		Missing Final State	5	21.74%
		Missing Composite State	5	21.74%
		Total	23	100.00%
ACT	Model Inaccuracy	Missing ActivityNode	8	29.63%
		Incorrect Node Placement	3	11.11%
		Incorrect Parallelism	3	11.11%
		Incorrect Use of Domain Terminology	2	7.41%
		Missing InitialNode	2	7.41%
	Over-specification	Extra ActivityNode	9	33.33%
		Total	27	100.00%
SD	Model Inaccuracy	Incorrect Messages type	45	45.00%
		Incorrect Messages interaction sequence	28	28.00%
		Incorrect CombinedFragment	13	13.00%
		Missing Messages	6	6.00%
		Missing Lifeline	5	5.00%
		Incorrect Lifeline	3	3.00%
			Total	100

The results indicate that requirement inconsistency hallucinations primarily arise from model inaccuracies, with some instances of over-specification. We identify three main contributing factors: (1) Insufficient understanding of requirements: LLMs may misinterpret key points, resulting in inaccurate models or irrelevant elements. (2) Increased error probability with model complexity: complex models require better context tracking and constraint handling, leading to higher error rates, particularly in SD tasks. (3) Limited logical reasoning: LLMs struggle to translate informal natural language requirements into formal models, especially in complex scenarios involving concurrency, conditional branches, and state transitions.

5.5 Findings in RQ1

The experimental results reveal substantial variation across model types. ACT performs the best, approaching human-level quality, while SD is the most challenging, with only about half of the interactions correctly generated. SD modeling requires a deeper understanding of message-passing sequences, which cannot be achieved through natural language comprehension alone—highlighting the need for additional support tools or structured guidance.

6 Results and Analysis of RQ2

Based on the findings of RQ1, we introduced model-checking rules to address the hallucinations identified in Tables 8–10. Each rule

targets a specific hallucination type and was incorporated into the Error section of the prompt template to enable feedback-driven model regeneration. We then measured the changes in T_G , Acc_p , Acc_s , and F1-score.

6.1 Model-Checking Rules Defined in Phase-II

Table 11 presents the model-checking rules for each hallucination type and their resolution rates in Phase-II. (1) For PlantUML Format Hallucinations, Non-Existent Element and Element Not Closed issues were fully resolved, while only half of the Format Errors were corrected. These are mostly syntactic and can be effectively addressed through rule-based validation. (2) For SysML Grammar Hallucinations, LLMs follow these guidelines as long as the model's syntactic specifications are clearly defined, significantly reducing syntax-related hallucinations. (3) For SysML Semantic Hallucinations, Some issues, such as incorrect ActivityFinalNode usage or missing ReduceAction, remained uncorrected. This highlights the difficulty of encoding deep semantic understanding into rule-based prompts, as current LLMs struggle with this level of abstraction. (4) For Requirements Inconsistency Hallucinations, resolution rates varied by model type: STM (42.14%), ACT (83.33%), and SD (16.67%). ACT errors were easier to fix, while SD errors, particularly those involving complex inter-object interactions, were the hardest to correct.

6.2 Model Generation Time in Phase-II

Figure 9 shows the model generation time in Phase-I and Phase-II. Since model-checking tasks for PlantUML format, SysML grammar, SysML semantics, and requirements consistency are performed sequentially, model regeneration occurs multiple times. The total generation time is calculated by summing the time for all regeneration tasks. For STM, ACT and SD, the (minimal ratio, maximum ratio) of G_{T-II} and G_{T-I} are (2.72, 7.67), (4.05, 4.81), (4.00, 4.31), respectively. This indicates that model generation with model-checking feedback enhancement is indeed time-consuming.

6.3 Accuracy and F1-Score in Phase-II

Figure 10 shows the changes in Acc_p , Acc_s , and F1 after applying the model-checking rules. For Acc_p , STM and SD generation achieved 100% across all models. This confirms the effectiveness of the rules in resolving format-related hallucinations. For Acc_s , most models reached near-100% accuracy in STM and ACT. However, variation persisted in SD: GPT-4o dropped to 93.11%, while GPT-4 and Kimi achieved 100% and 83.36%, respectively, indicating that these rules are insufficient to fully address SD-specific grammar issues. For F1-score improvements, all models showed substantial gains in STM and SD tasks. For instance, GPT-4o's F1-score improved from 49.79% to 74.11% for SD. Notably, Kimi and Claude exhibited the most significant improvements in SD, demonstrating the overall impact of model-checking feedback on semantic quality.

6.4 Findings in RQ2

The results show that model-checking rules effectively correct syntactic hallucinations, addressing structural and grammar errors and ensuring LLMs follow explicit grammar rules. However, semantic hallucinations in SysML have a low resolution rate of 36.89%,

highlighting LLMs' difficulty with SysML semantics and reasoning. For requirement consistency, ACT achieves an 83.33% resolution rate, STM 42.14%, and SD only 16.67%. Model-checking's impact on complex reasoning is limited. As rule complexity increases, LLMs may lose focus on the original requirements, and fixing one issue can introduce new ones.

One key reason is the lack of an optimal model-checking feedback mechanism in our study. To overcome these limitations, formal verification techniques present a promising solution. By automatically verifying model correctness, they can detect errors and generate counterexamples. These counterexamples can be provided as feedback to the LLM, aiding in refining its output and improving focus during regeneration. However, we have also noted that the cost associated with model checking is relatively high. Due to theoretical constraints inherent in model checking, verifying a specific property or functionality may require substantial time, and in some cases, may not yield a conclusive verification result.

7 Threat to Validity

Threats to Construct Validity stem from our goal of evaluating LLMs' performance in generating SysML behavioral models. To ensure an accurate assessment, the dataset must capture the complexity of SysML behavioral modeling. We addressed this by collecting a large, diverse dataset, including models from industrial providers. **Threats to Internal Validity** stem from our research design. The first threat is the reconstruction of behavioral model requirements. For models without explicit requirements, we manually created them, which is subjective. To reduce bias, we cross-checked the reconstructed requirements for consistency with the models. The second threat concerns LLM configuration. Model performance can vary with different inference frameworks. To control this, we used a greedy decoding strategy and set the temperature to 0, minimizing randomness and ensuring consistent behavior. **Threats to External Validity** relate to the generalizability of our findings. The first threat concerns the representativeness of our dataset. To address this, we gathered a diverse set of SysML behavior models from various domains, including rail transit, aerospace, and equipment manufacturing, ensuring broad real-world coverage. Although this paper has constructed a SysML model dataset spanning multiple domains, due to data sensitivity and confidentiality issues in safety-critical domains, the number of models available in certain fields is limited. Future research should further expand the sample size within each domain or collaborate with industry to obtain a wider range of real-world cases, thereby enhancing the generalizability of the research conclusions. The second threat is our selection of LLMs. We chose widely used models from leading companies and open-source communities, all with accessible APIs for practical use and integration in real-world applications.

8 Conclusion

Our work shows that when LLMs are given PlantUML format specifications and SysML syntax rules, their syntactic accuracy exceeds 90%. By providing requirement descriptions and additional data format information, LLM-generated models in ACT tasks show strong consistency with human-authored models. We summarize

Table 11: Model-checking Rules and Regeneration Results

Hallucinations Type	Hallucination Classification	Checking Rules	Resolved
PlantUML Format	Non-Existent Element	Do not use non-existent elements	30/30
	Format Error	The usage of elements comply with data format	2/4
	Element is not Closed	Paired elements must be closed	3/3
SysML Grammar	SD use Block syntax error	Use Block instead of Actor	16/18
	STM use Transition syntax error	Transition must connect two State	3/4
	STM use Composite State syntax error	Composite State must contain at least one Region.	2/2
SysML Semantics	ACT use Fork syntax error	Fork should contain at least two branches	1/1
	STM missing Regions	Region contains at least one InitialState and State	13/20
	STM missing PseudoState	STM must contain InitialState and FinalState	1/5
Requirements inconsistency	ACT use normal nodes as ActivityFinalNode	ActivityFinal is a special node	0/1
	ACT missing ReduceActions	Use ReduceAction to express loops	0/1
	ACT use meaningless ActivityPartitions	No explicit prompt not to use ActivityPartitions	1/1
	SD missing DestructionOccurrenceSpecification	The termination of Message should use DestructionOccurrenceSpecification	16/44
	STM Missing State or Transition	Don't miss out State in requirements	2/7
	STM Incorrect Transition	Pay attention to the rationality of the State between Traditions	0/6
	STM Missing Final State	Don't miss out Final State	3/5
	STM Missing Composite State	Use composite states for complex state	4/5
	ACT Missing ActivityNode	Don't miss out Activity in requirements	8/8
	ACT Incorrect Node Placement	Follow SysML layout and flow	3/3
	ACT Incorrect Parallelism	Use Fork/Join/concurrent regions correctly.	0/3
	ACT Incorrect Use of Domain Terminology	Use precise domain terms in requirements	2/2
	ACT Missing InitialNode	Don't miss out InitiaNode	2/2
	ACT Extra ActivityNode	Don't over-design	9/9
	SD Incorrect Messages type	The message type should meet the requirements and spec	12/45
	SD Incorrect Messages interaction sequence	Ensure correct message order.	8/28
	SD Incorrect CombinedFragment	CombinedFragment should meet the requirements and spec	1/13
	SD Missing Messages	Include all necessary messages.	2/6
	SD Missing Lifeline	Include all correct lifelines.	0/5
	SD Incorrect Lifeline	Include all correct lifelines.	0/3

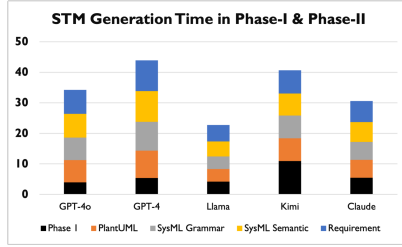
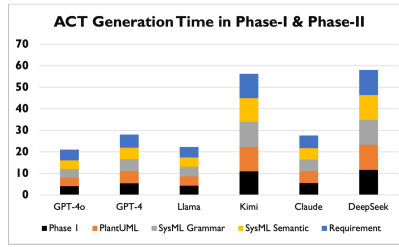
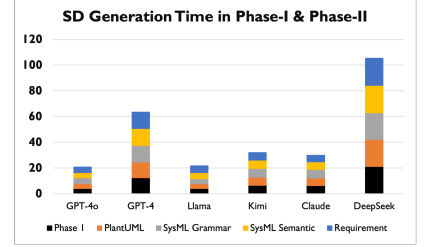
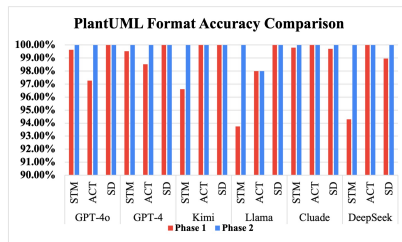
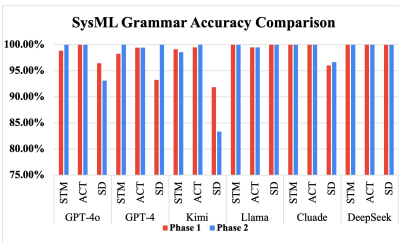
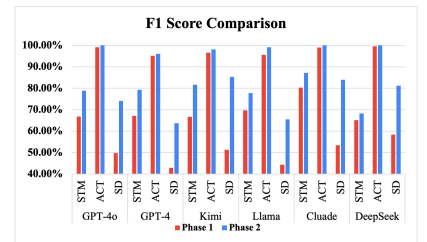

 (a) G_T of STM

 (b) G_T of ACT

 (c) G_T of SD

Figure 9: Model generation time in Phase-1 & Phase-II

 (a) Acc_P results

 (b) Acc_S results


(c) F1-score results

Figure 10: Syntactic and Semantic Accuracy with model-checking rules

LLMs' hallucinations during behavioral model generation and propose a feedback mechanism using model-checking rules to address them. While syntactic errors are mostly resolved, semantic issues persist, suggesting that the feedback mechanism using formal verification techniques needs further investigation to help LLMs better understand complex requirements and logical relationships.

Acknowledgments

This work was supported by National Key Research and Development Program of China under grant 2022YFB4501901, and by the State Key Laboratory of Complex & Critical Software Environment under grant CCSE-2024ZX-16.

References

- [1] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. 2023. Towards Human-Bot Collaborative Software Architecting with ChatGPT. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '23)*. ACM, Oulu, Finland, 279–285. doi:10.1145/3593434.3593468
- [2] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29.
- [3] Ludovic Apvrille and Bastien Sultan. 2024. System Architects Are not Alone Anymore: Automatic System Modeling with AI. In *12th International Conference on Model-Based Software and Systems Engineering*. SCITEPRESS-Science and Technology Publications, 27–38.
- [4] Ludovic Apvrille and Bastien Sultan. 2024. System Architects Are not Alone Anymore: Automatic System Modeling with AI. In *12th International Conference on Model-Based Software and Systems Engineering*. Rome, Italy, 27–38. doi:10.5220/0012320100003645
- [5] Enterprise Architect. 2024. Enterprise Architect. <http://www.sparxsystems.com>
- [6] Elias Bader, Dominik Vereno, and Christian Neureiter. 2024. Facilitating User-Centric Model-Based Systems Engineering Using Generative AI. In *Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering (MODELSWARD 2024)*. 371–377. doi:10.5220/0012623200003645
- [7] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*. 65–72.
- [8] Abdelhakim Baouya, Djamal Bennour, Otmame Ait Mohamed, and Samir Ouchani. 2015. A probabilistic and timed verification approach of SysML state machine diagram. In *2015 12th International Symposium on Programming and Systems (ISPS)*. IEEE, 1–9.
- [9] Nauman Bashir, Muhammad Bilal, Misbah Liaqat, Mohsen Marjani, Nadia Malik, and Mohsin Ali. 2021. Modeling class diagram using nlp in object-oriented designing. In *2021 National Computing Colleges Conference (NCCC)*. IEEE, 1–6.
- [10] Xiaohan Cao. 2021. *Modeling and Simulation of Deformation and Performance Coupling of Airborne Microstrip Antenna Array Based on SysML*. Master's thesis. University of Electronic Science and Technology of China.
- [11] M. Chami, C. Zoghbi, and J.M. Bruel. 2019. A First Step Towards AI for MBSE: Generating a Part of SysML Models from Text Using AI. *ResearchGate* (2019).
- [12] Mohammad Chami, Christophe Zoghbi, and Jean-Michel Bruel. 2019. A first step towards AI for MBSE: generating a part of SysML models from text using AI. *A First Step towards AI* (2019).
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [14] Shu CHEN. 2023. *Research and Implementation of a Safety-SysML/SCADE/AADL/Simulink Modeling Toolchain for Avionics Systems*. Master's thesis. East China Normal University.
- [15] Aaron Conrardy and Jordi Cabot. 2024. From Image to UML: First Results of Image-Based UML Diagram Generation using LLMs. *arXiv preprint arXiv:2404.11376v2* (Jun 2024). arXiv:2404.11376v2 [cs.SE]
- [16] Javier Cámara, Javier Troya, Lola Burgueño, and Antonio Vallecillo. 2023. On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. *Software and Systems Modeling* 22, 2 (2023), 781–793. doi:10.1007/s10270-023-01105-5
- [17] Dassault. 2022. MagicDraw. <https://www.3ds.com/products-services/catia/products/no-magic/magicdraw/>
- [18] Daniele De Bari, Giacomo Garaccione, Riccardo Coppola, Marco Torchiano, and Luca Ardito. 2024. Evaluating Large Language Models in Exercises of UML Class Diagram Modeling. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 393–399.
- [19] Deva Kumar Deeptimahanti and Muhammad Ali Babar. 2009. An automated tool for generating UML models from natural language requirements. In *2009 IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 680–682.
- [20] S. Dharmapuram, S. Ganipaneni, R.P. Kshirsagar, and O. Goel. 2025. Leveraging Generative AI in Search Infrastructure: Building Inference Pipelines for Enhanced Search Results. *ResearchGate* (2025). https://www.researchgate.net/profile/Suraj-Dharmapuram-2/publication/388659859_Leveraging_Generative_AI_in_Search_Infrastructure_Building_Inference_Pipelines_for_Enhanced_Search_Results/links/67a198fe207c0c20fa76804d/Leveraging-Generative-AI-in-Search-Infrastructure-Building-Inference-Pipelines-for-Enhanced-Search-Results.pdf
- [21] C. Dongmo. 2024. A Review of Non-Functional Requirements Analysis Throughout the SDLC. *Computers* (2024). https://www.researchgate.net/profile/Cyrille-Dongmo/publication/386120957_A_Review_of_Non-Functional_Requirements_Analysis_Throughout_the_SDLC/links/6746f616359dcb4d9d3ba59c/A-Review-of-Non-Functional-Requirements-Analysis-Throughout-the-SDLC.pdf
- [22] Eclipse. 2024. Papyrus. <https://www.eclipse.org/papyrus/>
- [23] Alessio Ferrari, Sallam Abualhajjal, and Chetan Arora. 2024. Model Generation with LLMs: From Requirements to UML Sequence Diagrams. In *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*. IEEE, 291–300.
- [24] Gaphor. 2024. Gaphor. <https://gaphor.org>
- [25] Angelo Gargantini, Elvinia Riccobene, and Patrizia Scandurra. 2010. Combining formal methods and MDE techniques for model-driven system design and analysis. *International journal on advances in software* 3, 1 (2010).
- [26] Ning Ge, Arnaud Dieumegard, Eric Jenn, and Laurent Voisin. 2018. Correct-by-construction specification to verified code. *Journal of Software: Evolution and Process* 30, 10 (2018), e1959.
- [27] Tom Gelhausen and Walter F Tichy. 2007. Thematic role based generation of UML models from real world requirements. In *International Conference on Semantic Computing (ICSC 2007)*. IEEE, 282–289.
- [28] Hassan Gomaa. 2019. *Real-Time Software Design for Embedded Systems*. China Mochine Press.
- [29] Google. [n. d.]. Google Scholar. <https://scholar.google.com/>
- [30] B. Graics, A. Vörös, and S. Tonetta. 2024. Towards the Formal Verification of SysML v2 Models. *ACM Transactions on Software Engineering and Systems* (2024). <https://dl.acm.org/doi/abs/10.1145/3652620.3687820>
- [31] I. Graja, S. Kallel, and N. Guermouche. 2020. A comprehensive survey on modeling of cyber-physical systems. *Concurrency and Computation: Practice and Experience* (2020).
- [32] Xu He, Haizhu Zhang, Guofu Ding, Rong Li, and Shuaihu Wang. 2022. Design and Modeling Method of High-Speed Train Bogie System Based on MBSE. 38, 04 (2022), 179–185+191. doi:10.13952/j.cnki.jofmndr.2022.0106
- [33] Yuqian HUANG, Baokun HU, Wanwan DIAO, and Zhenghua XIA. 2023. SysML-Based Monitoring and Simulation Research for Combat Scenarios. *Journal of Hangzhou Normal University (Natural Science Edition)* 22, 1 (2023), 88–95. doi:10.19926/j.cnki.issn.1674-232X.2023.01.013
- [34] John Hutchinson, Mark Rouncefield, and Jon Whittle. 2011. Model-driven engineering practices in industry. In *Proceedings of the 33rd International Conference on Software Engineering*. 633–642.
- [35] IBM. 2024. IBM Engineering Systems Design Rhapsody. <https://www.ibm.com/developerworks/downloads/r/rhapsodydeveloper/>
- [36] Mohd Ibrahim and Rodina Ahmad. 2010. Class diagram extraction from textual requirements using natural language processing (NLP) techniques. In *2010 Second International Conference on Computer Research and Development*. IEEE, 200–204.
- [37] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.
- [38] Fuyuan Jia. 2020. *Modeling Research and Verification of Airborne Communication System Based on MBSE*. Master's thesis. University of Electronic Science and Technology of China.
- [39] Dongming Jin, Shengxin Zhao, Zhi Jin, Xiaohong Chen, Chunhui Wang, Zheng Fang, and Hongbin Xiao. 2024. An Evaluation of Requirements Modeling for Cyber-Physical Systems via LLMs. *arXiv preprint arXiv:2408.02450* (2024).
- [40] R. Jordao. 2024. Modular and Tuneable Design Space Exploration in Model-Driven Engineering of Embedded Systems. *DiVA Portal* (2024). <https://www.diva-portal.org/smash/get/diva2:1901428/FULLTEXT01.pdf>
- [41] Deeptimahanti Deva Kumar and Ratna Sanyal. 2008. Static UML model generator from analysis of requirements (SUGAR). In *2008 Advanced Software Engineering and Its Applications*. IEEE, 77–84.
- [42] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [43] Yanli MA, Jizhong ZHANG, Jibin HAO, Wei LIANG, and Xiuchun LI. 2024. MBSE-Based Modeling Methodology for Diesel Engine System Design. *Journal of Graphics* 45, 2 (2024), 355–362.
- [44] Maziar Mahani, Denise Rizzo, Chris Paredis, and Yue Wang. 2021. Automatic formal verification of SysML state machine diagrams for vehicular control systems. *SAE International Journal of Advances and Current Practices in Mobility* 3, 2021-01-0260 (2021), 2272–2280.
- [45] Atif Mashkoor and Jean-Pierre Jacquot. 2011. Guidelines for formal domain modeling in Event-B. In *2011 IEEE 13th International Symposium on High-Assurance Systems Engineering*. IEEE, 138–145.
- [46] Microsoft. [n. d.]. GitHub. <https://github.com/>
- [47] Software Ideas Modeler. 2024. Software Ideas Modeler. <https://www.softwareideas.net/en/download>
- [48] Mustafa Abshir Mohamed, Moharram Challenger, and Geylani Kaldas. 2020. Applications of model-driven engineering in cyber-physical systems: A systematic mapping study. *Journal of computer languages* 59 (2020), 100972.
- [49] Priyanka More and Rashmi Phalnikar. 2012. Generating UML diagrams from natural language specifications. *International Journal of Applied Information Systems* 1, 8 (2012), 19–23.
- [50] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the*

- IEEE/ACM 46th International Conference on Software Engineering. 1–13.
- [51] NVIDIA. [n. d.]. Llama3.1:70B API. https://build.nvidia.com/nvidia/llama-3_1-nemotron-70b-instruct
- [52] Chinese Academy of Sciences. [n. d.]. China National Knowledge Infrastructure. <https://www.cnki.net/>
- [53] OMG. 2024. SysML-Tutorial. <https://sysml.org/tutorials/sysml-example-tutorial/>
- [54] OMG. November 2019. SysML1.6. <https://www.omg.org/spec/SysML/1.6/PDF>
- [55] Visual Paradigm. 2024. Visual Paradigm. <http://www.visual-paradigm.com/>
- [56] plantuml. [n. d.]. PlantUML. <https://plantuml.com/>
- [57] Maja Popović. 2015. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the tenth workshop on statistical machine translation*. 392–395.
- [58] Wen QI. 2022. *Research on SysML-Based Modeling Methodology for Manufacturing Equipment*. Master's thesis. Southwest Jiaotong University.
- [59] C.A. Ramirez and A.E. Thompson. 2024. A Design for Testability (DFT) Strategy for the Development of Highly Complex Safety-Critical System Using a Model-Based Systems Engineering (MBSE) Approach. In *IEEE Symposium on Systems Engineering*. <https://ieeexplore.ieee.org/abstract/document/10741151/>
- [60] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).
- [61] D. Ruddell, T. Biggs, and T. Lanigan. 2024. Modeling a Heavy-duty Vehicle Data Collection Process for Authenticating Driver Identity and Analyzing Driver Behavior Under Duress. In *IEEE System of Systems Conference*. <https://ieeexplore.ieee.org/abstract/document/10620958/>
- [62] Rick Steiner Sanford Friedenthal, Alan Moore. 2014. *A Practical Guide To SysML*. Morgan Kaufmann.
- [63] Douglas C Schmidt et al. 2006. Model-driven engineering. *Computer-IEEE Computer Society*- 39, 2 (2006), 25.
- [64] SinelaboreRT. 2025. SinelaboreRT. <https://www.sinelabore.de/doku.php/start>
- [65] Yifan Song, Guoyin Wang, Sujian Li, and Bill Yuchen Lin. 2024. The good, the bad, and the greedy: Evaluation of llms should not ignore non-determinism. *arXiv preprint arXiv:2407.10457* (2024).
- [66] Sara C Spangelo, David Kaslow, Chris Delp, Bjorn Cole, Louise Anderson, Elyse Fosse, Brett Sam Gilbert, Leo Hartman, Theodore Kahn, and James Cutler. 2012. Applying model based systems engineering (MBSE) to a standard CubeSat. In *2012 IEEE aerospace conference*. IEEE, 1–20.
- [67] Bastien Sultan and Ludovic Apvrille. 2024. Ai-driven consistency of sysml diagrams. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. 149–159.
- [68] Perla Tannoury and Ahmed Hammad. 2024. Integrating SysML and Timed Reo for Modeling Interactions in Cyber-Physical Systems Components. In *International Conference on Software Technologies*.
- [69] Perla Tannoury and Ahmed Hammad. 2024. Methodological Approach to Model and Validate CPS. In *International Conference on Simulation and Modeling Methodologies, Technologies and Applications*.
- [70] Christof Tinnes, Alisa Welter, and Sven Apel. 2025. Software Model Evolution with Large Language Models: Experiments on Simulated, Public, and Industrial Datasets. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 649–649.
- [71] Trufun. 2024. Trufun. <http://www.trufun.net/index.html>
- [72] Beian Wang, Chong Wang, Peng Liang, Bing Li, and Cheng Zeng. 2024. How LLMs aid in UML modeling: an exploratory study with novice analysts. In *2024 IEEE International Conference on Software Services Engineering (SSE)*. IEEE, 249–257.
- [73] Baomin Wang, Jiuhe Yang, Jinxiang Zhou, Haifang Wang, and Jun Che. 2024. Research on Traction Converter of Multiple Unit Train Based on MBSE. 06 (2024), 133–139. doi:10.13890/j.issn.1000-128X.2024.03.103
- [74] Yang WANG. 2021. *SysML-Based Design and Implementation of an Intelligent Guided Tour System for Botanical Gardens*. Master's thesis. University of Electronic Science and Technology of China.
- [75] Duo Wu. 2024. *Modeling and Analysis of Aluminum Extrusion Machine System Based on MBSE*. Master's thesis. Xi'an University of Technology.
- [76] M.D. Xames and T.G. Topcu. 2024. A Rapid Review of How Model-based Systems Engineering is Used in Healthcare Systems. *INCOSE International Symposium* (2024). <https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/iis2.13218>
- [77] Congying Xia, Chen Xing, Jiangshu Du, Xinyi Yang, Yihao Feng, Ran Xu, Wenpeng Yin, and Caiming Xiong. 2024. FOFO: A Benchmark to Evaluate LLMs' Format-Following Capability. *arXiv preprint arXiv:2402.18667* (2024).
- [78] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
- [79] Wenxing Xu. 2023. *Design of an Airborne Antenna Array Deformation Reconstruction and Electromagnetic Performance Coupled Simulation System Based on SysML*. Master's thesis. University of Electronic Science and Technology of China.
- [80] Jing YANG, Yangang LIANG, Kebo LI, and Yunchong ZHU. 2023. SysML-Based Architecture Design for Collaborative Combat of UAV Swarms. In *Proceedings of the 5th Conference on System Engineering: Systems Engineering in the Digital Intelligence Era*. College of Aerospace Science and Engineering, National University of Defense Technology; Hunan Provincial Key Laboratory of Intelligent Planning and Simulation for Aerospace Missions, 49–55.
- [81] Jia Zhang and Shuqun Yang. 2024. Recommendations for the Model-Based Systems Engineering Modeling Process Based on the SysML Model and Domain Knowledge. *Applied Sciences* 14, 10 (2024), 4010.
- [82] Tingda ZHAO. 2022. *Research on SysML and CPN-Based Integrated Train Control and Interlocking*. Master's thesis. Lanzhou Jiaotong University.